

Taal

The agent that sells what the forecast says you'll throw away.

TEAM DETAILS

Team name: Team Taal

Team leader: Nandish

PROBLEM STATEMENT

Grocery and quick-commerce retailers forecast demand in one tool and run offers in another. So stock is written off at one dark store while the same SKU runs out at the next, customers get blanket markdowns instead of offers matched to what they actually buy, and no forecast plans to the cut-off that matters — FSSAI's advisory that food sold online must keep 30% of its shelf life, or 45 days. Taal is one loop: forecast every SKU at every node, find the gap, plan the play, offer it to consenting customers, measure it against a holdout.

BRIEF ABOUT THE IDEA

One decision loop that turns a forecast into a sale, before the write-off happens.

- 1 Capture** — a node manager photographs a pallet; Gemini reads dates and counts with confidence scores
- 2 Sense** (nightly) — per-SKU, per-node forecasts surface gaps with rupees at stake and the online sell-by cut-off as the deadline
- 3 Plan** — an ADK Planner Agent designs a play; every rupee comes from a deterministic estimator; eight guardrails gate it
- 4 Approve** — a human approves; a holdout is assigned by hash; the play is written back into the forecast as a covariate
- 5 Engage** — a Customer Agent grounded in real stock and consent delivers vernacular offer copy with the best-before date disclosed
- 6 Measure** — treated vs. holdout with a 95% CI; the estimator's prior is updated from what was actually measured

ONE REAL EXAMPLE, SEEDED DEMO TENANT

6 days

until Masala Chips at dark store DS-07 can no longer be sold online
— 368 units, ₹9,200 at stake

The Desk shows a real Gemini run, recorded 28 Sep: the model rejected a transfer for negative margin, proposed a chips + coconut-water bundle at ₹58.50, had its rationale sent back three times by the cite-or-drop guardrail, and passed all eight on the fourth attempt. Approve assigns 316 treated / 36 holdout by hash; the projected write-off moves ₹9,194 → ₹8,068 in one ~10 s call.

OPPORTUNITIES

Not a coupon bot, and not a forecast that stops at a number

How it differs

Most demand-forecasting tools stop at a number; most promo tools have no forecast behind them. Taal closes the loop: an approved play is written into **future_regressors** and the affected series is re-forecast immediately — the chart moves because a decision was made, not because the next nightly cycle caught up.

How it solves the problem

A deterministic estimator, never the model, owns every rupee. Eight guardrails (margin floor, frequency cap, consent, no cannibalising a stockout, and more) gate every play before it ships. A randomized holdout on every approval proves the play worked, instead of assuming it did.

USP

Every play ships with a randomized holdout baked in — not a chosen comparison, a held-back control group assigned by hash before the play ever runs. The Outcomes screen reports treated vs. holdout with a 95% CI, and marks a play **unmeasured** rather than fabricate a lift from too few responders. A promo tool with no forecast, or a forecast tool with no control group, cannot make that claim.

LIST OF FEATURES

Six agent-backed capabilities, gated by rules a model never touches



Vision intake

Photograph a pallet; SKU, best-before date, facings and confidence come back structured; low-confidence rows must be confirmed.



Nightly Sense

Per-SKU, per-node forecasts with promo and festival regressors; five grocery gap types, each with rupees at stake and a deadline.



Planner Agent

An ADK LoopAgent (max 3 iterations), six deterministic tools, a Cost Governor that decides which gaps are worth a model call.



Approve + assignment

Holdout assignment by hash, vernacular copy with a best-before disclosure, offers for treated customers only, live re-forecast.



Customer Agent

Grounded in real node stock, offers and consent; orders through an MCP endpoint; STOP withdraws consent immediately.

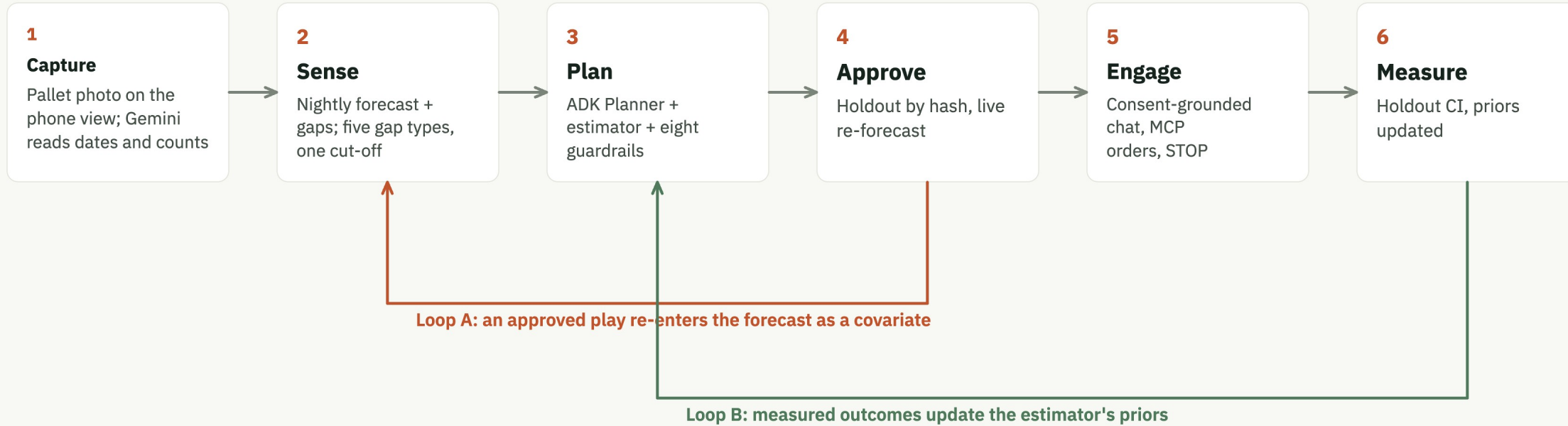


Measure

Treated vs. holdout inside the play window, 95% CI, estimator priors updated from real treated/responder counts.

PROCESS FLOW

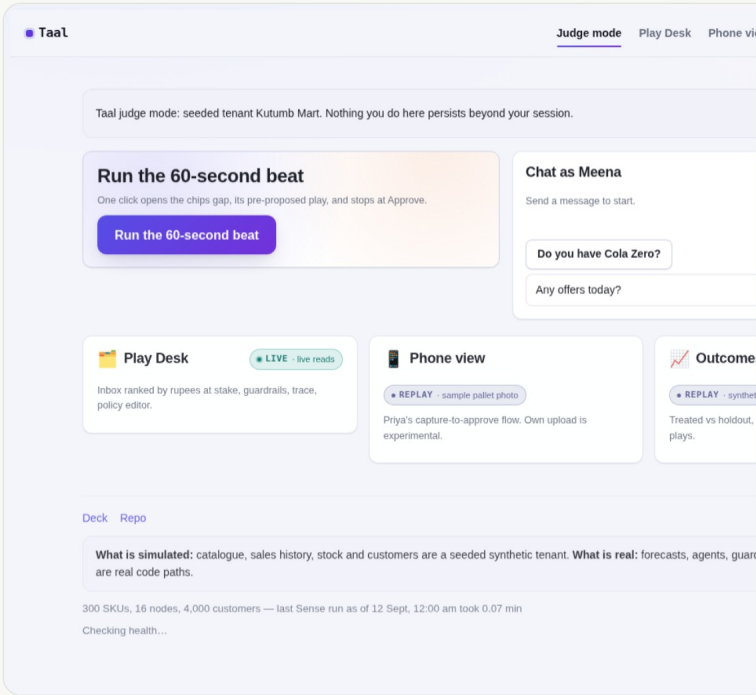
Two closed loops, not an open-ended pipeline



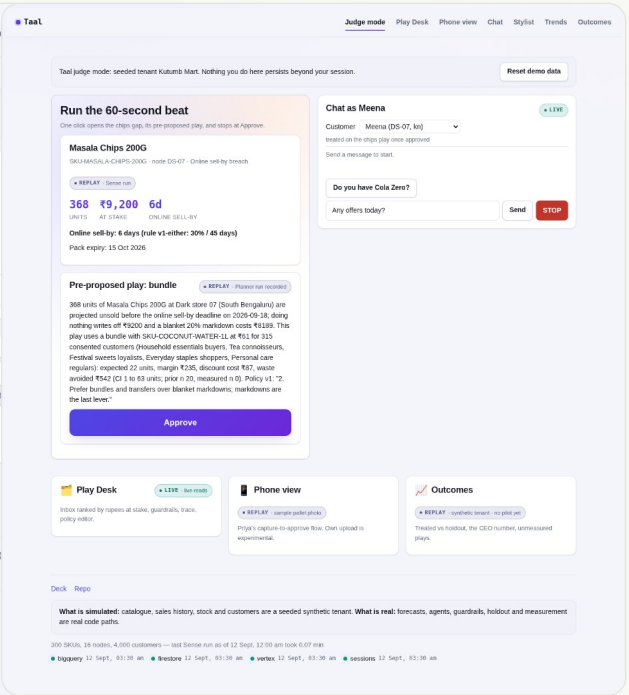
WIREFRAMES / MOCK DIAGRAMS (OPTIONAL)

Built directly as a working prototype — no separate wireframe stage

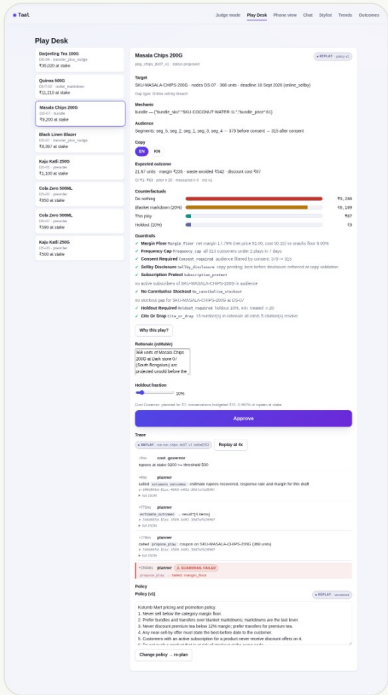
These are real screens of the running app (make api + make web), not mockups.



Judge-mode landing



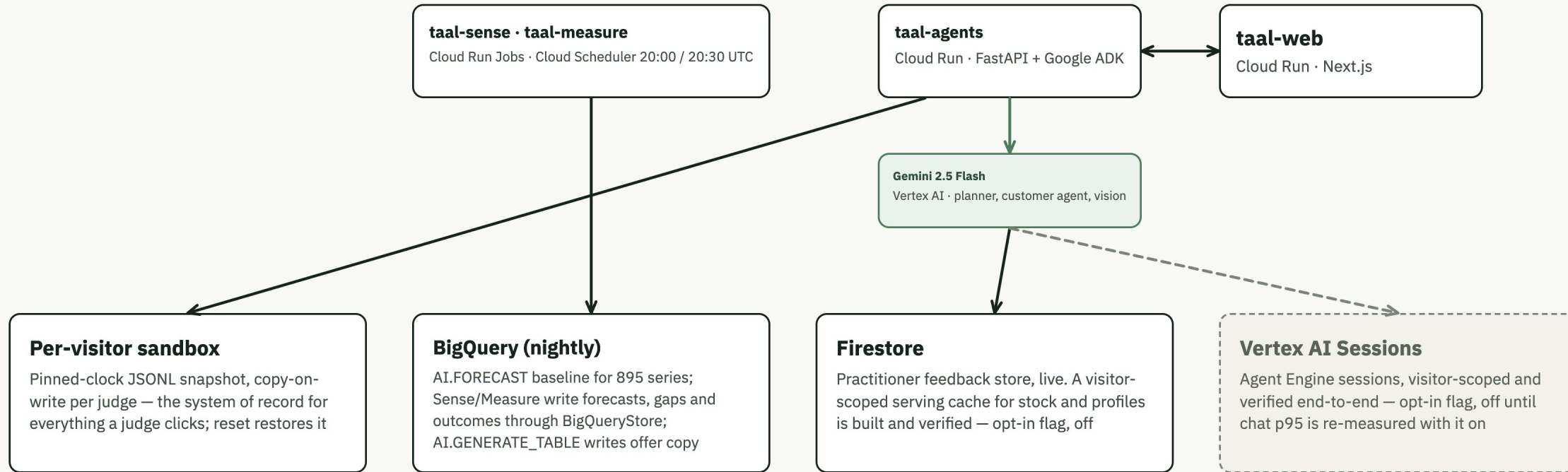
60-second beat: gap card



Play Desk

ARCHITECTURE

Two paths: a per-visitor sandbox serves judges; nightly jobs run on BigQuery



— Live in the deployed project today ---- Built and verified against real GCP — opt-in flag, not yet on

TECHNOLOGIES USED

Google Cloud AI stack, verified against real services

Gemini 2.5 Flash

On Vertex AI, via Google ADK. Vision intake, the Planner Agent, the Customer Agent — guardrail-gated throughout.

Google ADK

An LlmAgent inside a LoopAgent with six deterministic tools. A real 65 s planning run is recorded and seeded on the Desk; live re-plans stream every tool call and guardrail check.

BigQuery

AI.FORECAST (TimesFM) ran the nightly baseline for all 895 series — 25,060 rows in 23 s. ARIMA_PLUS_XREG verified on 20 series; AI.GENERATE_TABLE writes the offer copy.

Firestore

Practitioner feedback store, live in the deployed service. A visitor-scoped serving cache is built and verified against a real Firestore Native database — opt-in.

Cloud Run + Cloud Scheduler

taal-agents and taal-web services that scale to zero, plus taal-sense and taal-measure Cloud Run Jobs fired nightly by Cloud Scheduler. Cloud Build deploys every push to main.

Next.js + FastAPI + MCP

Play Desk, phone view and chat on Next.js; the API on FastAPI; orders through a Model Context Protocol endpoint.

ESTIMATED IMPLEMENTATION COST (OPTIONAL)

Priced from real, measured usage — not a guess

TRAILING 30-DAY WINDOW, MEASURED

\$5.86 / ₹486

Priced against Google's public list rates from Cloud Monitoring: real Gemini token counts, real BigQuery bytes billed across 752 jobs, real Cloud Run CPU/memory allocation time.

Methodology: modeled_from_measured_usage — not a billing-account export, never conflated with an actual invoice.

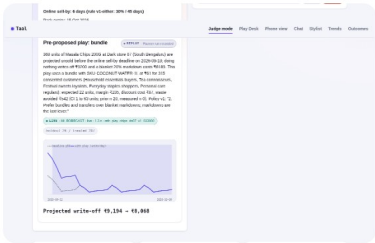
Forward-looking profile estimates

Profile	Scale	Est. monthly cost
Micro	1 shop, ≤500 SKUs	₹200–500
Mid (this demo's size)	300 SKUs, 10 nodes, ~4,000 customers	₹2,500–5,000
Large	20,000 SKUs, 500 nodes, 1M customers	₹1–2 lakh

Estimates from the design's cost-governance model (batching, caching, model tiering), not measured at these scales. The measured ₹486 sits below the Mid estimate because the demo carries judge traffic, not a tenant's nightly volume of plays and conversations.

SNAPSHOTS OF THE PROTOTYPE

The loop, mid-interaction — captured against the real running stack



Approved: the forecast chart moves

The screenshot shows a chat interface. The header says 'Chat as Meena' with a 'LIVE' status and '47 MS'. The user is 'Meena (DS-07, kn)'. The message says 'based on the chips play once approved'. Below the message is a button 'Any offers today?'. The message content is 'Masala Chips 200G - 1000 Coconut Water 1L star *61g 100, 10000 100000, 10000 15 Oct 2026 2026 2026'. Below the message are buttons 'Add to cart', 'Not now', and 'STOP'. The message ends with 'cled: play_chips_200g_v1'. Below the message is a form 'Do you have Cola Zero?' with a 'Send' button and a 'STOP' button. The form has a placeholder 'Type a message...'.

Meena's chat: Kannada offer, best-before shown

The screenshot shows a phone view interface. At the top is a section 'Own photo (experimental)' with a button 'Take or choose a photo'. Below this is a table with columns: 'SKU', 'GUESS', 'BEST BEFORE', 'FACINGS', and 'CONFIDENCE'. The table has one row of data: 'SKU- MASALA-CHIPS-200G', '2026-11-02', '2', and 'sku 100% - date 100% - count 100%'. Below the table is a button 'CVI'.

Phone view: pallet photo, intake table

The screenshot shows an outcomes interface. The header says 'Outcomes'. Below the header is a table with columns: 'Outcomes', 'Measured', 'Unmeasured', and 'Status'. The table has three rows of data: '0.4 kg', '1.0 kg', and '150'. Each row has a 'Measured' column with a value, an 'Unmeasured' column with a value, and a 'Status' column with a value. Below the table is a button 'CVI'.

Outcomes: measured vs. unmeasured plays

Real numbers, each traced to a committed raw output

Metric	Result	Context
Planner, live Gemini on the flagship gap	5/5 proposed, 0 fallbacks	34–65 s each; run 4 (65 s, 7 iterations) is the trace seeded on the Desk
Customer Agent latency (live Vertex, 28 Sep)	p50 4.56 s / p95 6.48 s	50/50 calls returned 200; p95 just over the 6 s target (5.75 s on 21 Sep)
Approve → holdout + re-forecast (live)	8.3–10.6 s	Two measurements; projected write-off 9,194 → 8,068
Agent Simulation pass rate	190/190 (100%)	Live Vertex, 5 customer-agent checks: stock, holdout arm, stacking, STOP, hallucination
BigQuery AI.FORECAST (TimesFM)	895 series, 25,060 rows, 22.7 s	One job; the local model still won 45/45 backtest comparisons on this seeded data
BigQuery ARIMA_PLUS_XREG	20 of 895 series	560 forecast rows written, over a 324,271-row sales load
Forecast backtest MAPE, 9 tiers	0.107–0.221	63 rows, 7 rolling origins each
Sense job throughput	3.83 s	300 SKUs × 16 nodes/outlets, 550 gaps produced
Vision intake accuracy	30/30 (100%)	Synthetic photos; the real-photo evaluation harness is built, photos pending

Every number is reproduced from a file under eval/raw/; metrics not yet measured are marked so in eval/evaluation_table.md rather than guessed.

What ten retail practitioners told us (counts, not percentages)

6 of 10

write off stock weekly or daily

4 of 8

online sellers have had stock blocked
by the online sell-by limits

1 of 10

runs any control group when a
promotion goes out

Taal makes the two missing things the default: the online sell-by cut-off is the deadline every forecast plans to, and every play ships with a hash-assigned holdout.

Collected through the product's own /feedback form; ten usable responses; contact details held separately and never published.

What's built and held back, and what's next

Built, verified, behind an opt-in flag

- ☁ Vertex AI Sessions — Agent Engine sessions scoped to the visitor so two judges' chats can never cross; off until chat p95 is re-measured with it on
- 🔒 Firestore serving cache — overlay-aware, so a visitor's own Approve is never served stale; off for the same reason
- 📋 ARIMA_PLUS_XREG on the approve path — SQL verified; approve still re-forecasts with the local model until per-visitor isolation is solved in BigQuery

Next up

- 🧪 A real pilot: consented customers, a pre-registered metric, a genuine holdout — designed in docs/pilot.md; the three-CSV ingest turns a shop's history into gaps in one command
- 🔍 Vision on real camera photos — the evaluation harness is built; 30 shelf photos with ground truth are the missing input
- ▶ Gemini Live voice intake — stubbed today, a real spoken confirmation flow for the phone view

LINKS

Taal — the agent that sells what the forecast says you'll throw away

 GITHUB PUBLIC REPOSITORY

 DEMO VIDEO (3 MINUTES)

[ADD VIDEO LINK — not yet recorded]

 FINAL PRODUCT LINK